Parametric Disjunctive Cuts for Sequences of Mixed Integer Linear Optimization Problems

Shannon Kelley[1], Aleksandr M. Kazachkov[2], and Ted Ralphs[1]

[1] Lehigh University, Bethlehem, PA, USA
[2] Universit of Florida, Gainesville, FL, USA

Abstract. Many applications require solving sequences of related mixed-integer linear programs. We introduce a class of *parametric disjunctive inequalities* (PDIs), obtained by reusing the disjunctive proofs of optimality from prior solves to construct cuts valid for perturbed instances. We describe several methods of generating such cuts that navigate the trade-off between computational expense and strength. We provide sufficient conditions under which PDIs support the disjunctive hull and a tightening step that guarantees support when needed. On perturbed instances from MIPLIB 2017, augmenting branch-and-cut with PDIs substantially improves performance, reducing total solve times on the majority of challenging cases.

Keywords: parametric optimization, warm starting, disjunctive cuts, MILP

1 Introduction

We propose a class of *parametric disjunctive inequalities* (PDIs) for accelerating the solution of sequences of mixed integer linear optimization problems (MILPs). There are many methodologies whose implementation requires solving sequences of instances from a parametric family of MILPs, including column generation, Lagrangian relaxation, Benders decompositions, mixed integer nonlinear optimization, bilevel optimization, multiobjective optimization, and online optimization [5, 6, 18, 13, 24, 14, 21]. Applications are diverse, including energy problems, supply chain optimization, and revenue management [25, 22, 16].

The similar structure of the instances in parametric families makes them natural candidates for warm starting. Existing approaches reuse solutions and disjunctions, transfer parameters and branching policies, guide cut selection [21, 17, 20, 13, 16, 11], or even re-use the entire branch-and-bound tree [17]. Yet, despite the central role of cutting planes in performance [8], the cut generation process itself is rarely warm started.

This omission is understandable: most cuts are inexpensive to regenerate, so parameterization can appear unnecessary. Two observations motivate a different view. First, cuts are typically generated with default settings, and recent work shows that learning instance-dependent parameters can improve solver performance [7, 10, 12]. Second, some families—especially disjunctive cuts from large

disjunctions—offer among the strongest relaxations [23, 9, 4], yet are often disabled in practice due to their computational cost [8, 2]. We address how to obtain strong disjunctive cuts without repeatedly solving the auxiliary LPs they typically require: our PDIs exploit shared structure to amortize generation across related solves, reducing time while retaining much of the original strength.

The validity of the disjunctions we construct is based solely on integrality of the variables, arising as the leaf nodes of (partial) *branch-and-bound* trees constructed using the standard strategy of branching on variables. The key insight behind parametric disjunctive inequalities is that the certificate of validity of an inequality generated from such a disjunction can be easily exploited to provide an inequality and a certificate of validity for *any* other MILP with the same number of constraints and the same integer variables. We focus on cuts derived from disjunctions with a large number of terms, which are expensive to generate but powerful. We show that parameterizing their certificates across a sequence of instances effectively amortizes generation cost. We then evaluate whether this accelerates overall solution time of structurally similar problems.

Contributions.

i) We introduce a class of PDIs for which we describe how to parameterize their certificates of validity and use these parametric certificates to generate inequalities valid for new instances.
ii) We prove conditions under which a PDI, when applied to a new instance, supports the disjunctive hull, as well as introduce a tightening procedure that ensures this property holds.
iii) We integrate the generation of PDIs into branch and cut and evaluate performance on families of instances derived by perturbing MIPLIB 2017 instances. We find consistent improvements over the generation procedure described in Balas and Kazachkov [4] (which is not parametric) and over a default baseline that does not use disjunctive cuts for challenging cases.

2 Notation and Preliminaries

Our goal is to generate inequalities valid for an MILP drawn from a parametric family $\mathcal{K}$ of related instances. While our techniques apply as long as the set of integer-constrained variables is consistent across instances, we assume for simplicity that all instances are defined over the same variables and number of constraints.

Each instance $k \in \mathcal{K}$ takes the form:

$$\min_{x \in \mathcal{S}^k} \quad c^k x \tag{IP-k}$$

where $\mathcal{S}^k := \left\{x \in \mathcal{P}^k : x_j \in \mathbb{Z} \text{ for all } j \in \mathcal{I}\right\}$ and $\mathcal{P}^k := \left\{x \in \mathbb{R}^n : A^k x \geq b^k\right\}$, where $A^k \in \mathbb{Q}^{q \times n}$, $b^k \in \mathbb{Q}^q$, $c^k \in \mathbb{Q}^{1 \times n}$, and $\mathcal{I} \subseteq [n] := \{1, \ldots, n\}$ is the set of indices of integer variables. We assume the constraints include nonnegative

upper and lower bounds on all variables. For matrix A, we represent its i^{th} row as $A_{i\cdot}$ and its j^{th} column as $A_{\cdot j}$.

To strengthen the LP relaxation, we rely on *disjunctive cuts*, which are valid inequalities derived from valid disjunctions.

Definition 1. *A* valid inequality *for a set $\mathcal{S} \subseteq \mathbb{R}^n$ is a pair $(\alpha, \beta) \in \mathbb{R}^n \times \mathbb{R}$ such that $\mathcal{S} \subseteq \left\{x \in \mathbb{R}^n : \alpha^\top x \geq \beta\right\}$.*

Definition 2. *A* valid disjunction *for a set $\mathcal{S} \subseteq \mathbb{R}^n$ is a collection $\mathcal{X} := \{\mathcal{X}^t\}_{t \in T}$, where T is an index set, $\mathcal{X}^t \subset \mathbb{R}^n$ for $t \in T$, and $\mathcal{S} \subseteq \mathcal{X}$.*

In this work, we assume $\mathcal{X}^t := \{x \in \mathbb{R}^n : D^t x \geq D_0^t\}$, where $D^t \in \mathbb{Q}^{q_t \times n}$ and $D_0^t \in \mathbb{Q}^{q_t}$. In fact, we further assume $\mathcal{X}^t$ is a hyperrectangle obtained by restricting the bounds on one or more variables.

We frequently reference the feasible region of an instance $k \in \mathcal{K}$ restricted to a disjunctive term $t \in T$:

$$A^{kt} := \begin{bmatrix} A^k \\ D^t \end{bmatrix}, \quad b^{kt} := \begin{bmatrix} b^k \\ D_0^t \end{bmatrix}, \quad \text{and} \quad \mathcal{Q}^{kt} := \mathcal{P}^k \cap \mathcal{X}^t = \left\{x \in \mathbb{R}^n : A^{kt} x \geq b^{kt}\right\}.$$

Additionally, we refer to the convex hull of feasible regions defined by all terms of a disjunction as the *disjunctive hull*, defined as $\mathcal{P}_D^k := \operatorname{cl}\operatorname{conv}\left(\bigcup_{t \in T} \mathcal{Q}^{kt}\right)$, and we index the subset of disjunctive terms that are feasible for IP-k as $T_{\text{feas}}^k := \{t \in T : \mathcal{Q}^{kt} \neq \emptyset\}$.

We refer to the set of indices of any collection of n linearly independent constraints binding at some basic solution as a *basis*. Conceptually, this is equivalent to the concept of a basis that arises in the theory of LPs, when the problem is usually assumed to be in standard form. Bases in standard form are in one-to-one correspondence to what we are here calling a basis and as such, they can be feasible or infeasible, as specified in the following definition.

Definition 3. *A basis $\mathcal{A}^t$ is* feasible *for $\mathcal{Q}^{kt}$ if the associated basic solution $(A^{kt}_{\mathcal{A}^t \cdot})^{-1} b^{kt}_{\mathcal{A}^t}$ lies in $\mathcal{Q}^{kt}$.*

Disjunctive Cut Generation. Valid inequalities for the disjunctive hull $\mathcal{P}_D^k$ can be generated by solving an LP, either the classical *Cut Generating LP (CGLP)* [3, 9] or the so-called *Point-Ray LP (PRLP)* [23, 4]. Regardless of which of these two LPs is used, the solution is an inequality $(\alpha, \beta) \in \mathbb{R}^n \times \mathbb{R}$ that satisfies:

$$\left.\begin{array}{l} \alpha^\top = v^t A^{kt} \\ \beta \leq v^t b^{kt} \\ v^t \in \mathbb{R}_{\geq 0}^{1 \times (q + q_t)} \end{array}\right\} \text{ for all } t \in T. \tag{FL}$$

By *Farkas' Lemma* [15], this condition is equivalent to the inequality (α, β) being valid for $\mathcal{P}_D^k$. We refer to $\{v^t\}_{t \in T}$ as the *Farkas multipliers* or *Farkas certificate* for (α, β) relative to the disjunction $\mathcal{X}$ and instance IP-k.

The method used to compute $\{v^t\}_{t \in T}$ depends on whether we are solving a CGLP or a PRLP. In the CGLP, condition (FL) is included directly in the

formulation, making $\{v^t\}_{t\in T}$ part of the solution. In contrast, the PRLP reduces the dimensionality of the problem by eliminating the variables whose values produce the Farkas certificate [23], allowing for more efficient computation with larger disjunctions and, consequently, stronger cuts [4]. For this reason, we adopt the PRLP when generating disjunctive cuts from an LP. The corresponding Farkas certificate is then reconstructed post hoc, as described by Kazachkov and Balas [19, Lemma 3]. In practice, each term v^t is computed from $\mathcal{A}^t$ and $\mathcal{Q}^{kt}$, selected to represent the closest feasible basis to the cut (α, β) for term t. If $\mathcal{Q}^{kt}$ is infeasible and no such basis exists, we assign $v^t = 0$ by default.

3 Parametric Disjunctive Cuts

Certificates. The certificate of validity for a PDI consists of a disjunction and a set of multipliers (the Farkas certificate) associated with each term of the disjunctions that together satisfy (FL), certifying validity of the cut.

Definition 4. *Let $k \in \mathcal{K}$, $\{v^t\}_{t\in T}$, and a disjunction $\{\mathcal{X}^t\}_{t\in T}$ valid for* IP-k *be given. Then $(\{v^t\}_{t\in T}, \{\mathcal{X}^t\}_{t\in T})$ is a* certificate of validity *for $(\alpha^\top, \beta)$ if $\alpha_j := \max_{t\in T}\{v^t A^{\ell t}_{\cdot j}\}$ and $\beta := \min_{t\in T}\{v^t b^{\ell t}\}$ for all $j \in [n]$.*

In our algorithms, certificates of validity are constructed with respect to one particular instance in a family and then are used to generate cuts for other instances.

Definition 5. *An MILP instance* IP-k *and disjunction $\{\mathcal{X}^t\}_{t\in T}$* induce *the Farkas certificate v^t if $v^{t_1} A^{kt_1} = v^{t_2} A^{kt_2}$ for all $(t_1, t_2) \in \{\mathcal{X}^t\}_{t\in T^k_{feas}} \times \{\mathcal{X}^t\}_{t\in T^k_{feas}}$.*

To identify the basis used to construct the Farkas coefficients for a given feasible disjunctive term $\mathcal{Q}^{kt}$, we associate v^t with $\mathcal{A}^t$, a basis of $\mathcal{Q}^{kt}$, which contains the indices of the constraints whose linear combination produces the cut coefficients α.

Definition 6. *The basis $\mathcal{A}^t$* determines *the Farkas certificate v^t if $\{i \in [q + q_t] : v^t_i > 0\} \subseteq \mathcal{A}^t$.*

Parametric Cut Generation. To reuse disjunctive inequalities across related instances while preserving validity, we derive parametric inequalities (parametric cuts) for the disjunctive relaxation via a procedure that guarantees their validity by ensuring such validity is satisfied by a known certificate.

Definition 7. *Let $\mathcal{K}$ be the indices of a family of MILPs, $\{\mathcal{X}^t\}_{t\in T}$ be a disjunction valid for every member of the family, and $\{v^t\}_{t\in T}$ be a Farkas certificate satisfying* (FL). *Then $\{(\alpha^k, \beta^k)\}_{k\in\mathcal{K}}$ is a* family of parametric disjunctive inequalities (PDIs) *with respect to $(\{v^t\}_{t\in T}, \{\mathcal{X}^t\}_{t\in T})$ if for all $k \in \mathcal{K}$, we have that $(\{v^t\}_{t\in T}, \{\mathcal{X}^t\}_{t\in T})$ is a certificate of validity of (α^k, β^k) for $\mathcal{S}^k$.*

Validity. We now establish procedures for generating parametric families of valid inequalities (proofs of correctness in Appendix A) by taking a previously generated certificate of validity as input and generating a cut satisfying (FL) from it. By taking the Farkas certificate as an input, we avoid solving solving an LP to compute it, while hopefully retaining the strength of the cuts due to the similarity between the instances.

Definition 8. *A* family of Farkas PDIs *is* $\{(\alpha^k, \beta^k)\}_{k\in\mathcal{K}}$ *such that* $(\alpha^k, \beta^k) \coloneqq ([\max_{t\in T}\{v^t A^{kt}_{\cdot 1}, ..., \max_{t\in T}\{v^t A^{kt}_{\cdot n}\}]^\top, \min_{t\in T}\{v^t b^{kt}\})$.

For ease of exposition, when we describe families of PDIs going forward, we mean *Farkas* PDIs. The following result shows they are parametrically valid for $\mathcal{P}^\ell_D$. For clarity, we use the indices k and ℓ to denote, respectively, the instance that induces the Farkas certificate and the instance to which it is applied.

Theorem 1. *Let* $k, \ell \in \mathcal{K}$. *Then given a Farkas certificate* $\{v^t\}_{t\in T}$ *induced by disjunction* $\{\mathcal{X}^t\}_{t\in T}$ *and* IP-k, *we have that* $\alpha^\top x \geq \beta$ *for all* $x \in \mathcal{P}^\ell_D$, *where* $\alpha_j \coloneqq \max_{t\in T}\{v^t A^{\ell t}_{\cdot j}\}$ *and* $\beta \coloneqq \min_{t\in T}\{v^t b^{\ell t}\}$ *for all* $j \in [n]$.

Figures 1a and 1b illustrate cuts produced by Theorem 1, marked in the figures with the annotations "2) generate parametric disjunctive cut".

Strengthening. A natural question arises when generating a PDI (α, β) for a disjunction $\{\mathcal{X}^t\}_{t\in T}$ and instance IP-k: does the cut support the disjunctive hull $\mathcal{P}^k_D$?

Definition 9. *The inequality* (α, β) *valid for a polyhedron* $\mathcal{P}$ supports $\mathcal{P}$ *if there exists* $x \in \mathcal{P}$ *such that* $\alpha^\top x = \beta$.

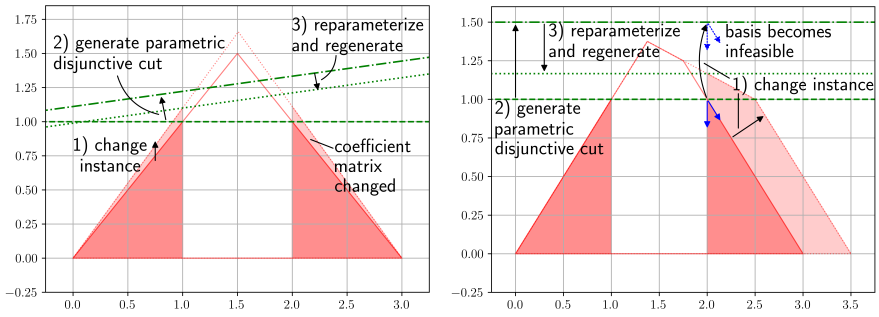
Cuts generated by solving the system (FL) are guaranteed to support the disjunctive hull, but PDIs from Theorem 1 need not. The following conditions guarantee support, however.

Lemma 1. *Let* $k, \ell \in \mathcal{K}$; $\{v^t\}_{t\in T}$ *be a Farkas certificate induced by* IP-k *and disjunction* $\{\mathcal{X}^t\}_{t\in T}$; $\{(\alpha^k, \beta^k)\}_{k\in\mathcal{K}}$ *be the associated family of Farkas PDIs; and* $\mathcal{A}^t$ *be the basis determining* v^t *for all* $t \in T$. *Then* $(\alpha^\ell, \beta^\ell)$ *supports* $\mathcal{P}^\ell_D$ *if*

1. $\{v^t\}_{t\in T}$ *is induced by* IP-ℓ *and* $\{\mathcal{X}^t\}_{t\in T}$, *and*
2. $\mathcal{A}^t$ *is feasible for* $\mathcal{Q}^{\ell t}$ *for all* $t \in T$.

Condition 1 fails when the coefficient matrix changes or a term that is infeasible in IP-k becomes feasible in IP-ℓ. Condition 2 breaks when $\mathcal{A}^t$ becomes infeasible for some $\mathcal{Q}^{\ell t}$ when it was feasible for IP-k. The latter is often mitigated by removing infeasible terms before parameterization, leaving only bases that turn infeasible for still-feasible terms. See "2)" in Figures 1a and 1b below and the emergence of a newly feasible term in Figure 5 in Appendix C.

The conditions of Lemma 1 hint at a method of recovering disjunctive hull support when it is lost. We recover support in two steps: (1) *reparameterize* by computing the certificate for each term as needed, then (2) *regenerate* the PDI using the updated certificates. The result of these steps are annotated "3)" in each of the three failure cases illustrated Figures 1a, 1b, and 5.



(a) matrix perturbation (b) infeasibility from perturbation

Fig. 1: Examples showing that PDIs from the same certificate need not all support the disjunctive hull

Step 1: Reparameterize to Find Supporting Certificates. We refer to the process of obtaining a supporting cut and its corresponding *supporting certificate*—the Farkas certificate that generates the supporting cut—as *reparameterization.* Lemma 2 shows that this involves solving the primal-dual pair LP-$\ell t(\alpha)$ and Dual-$\ell t(\alpha)$

$$\min_{x \in \mathcal{Q}^{\ell t}} \ \alpha^\top x \qquad \text{(LP-}\ell t(\alpha)\text{)} \qquad\qquad \max_{v \in \mathcal{D}^{\ell t}(\alpha)} \ v b^{\ell t} \qquad \text{(Dual-}\ell t(\alpha)\text{)}$$

where $\mathcal{D}^{\ell t}(\alpha) := \{v \in \mathbb{R}^{1 \times (q+q_t)} : vA^{\ell t} = \alpha^\top,\ v \geq 0\}$.

Lemma 2. *Let $\ell \in \mathcal{K}$, $t \in T$, and $\alpha \in \mathbb{R}^n$. If $\mathcal{Q}^{\ell t}$ is bounded and nonempty, then $(\alpha, \alpha^\top \bar{x})$ such that $\bar{x} \in \arg\min_{x \in \mathcal{Q}^{\ell t}} \{\alpha^\top x\}$ is a supporting inequality for $\mathcal{Q}^{kt}$ with supporting certificate $\bar{v}^t \in \arg\max_{v^t \in \mathcal{D}^{\ell t}(\alpha)} \{v^t b^{\ell t}\}$.*

Further, provided there is no constraint in $\mathcal{Q}^{\ell t}$ parallel to α, Corollary 1 shows that the only supporting cut with coefficients α and a nonnegative Farkas certificate is the one derived from the optimal solutions to the primal–dual pair LP-$\ell t(\alpha)$ and Dual-$\ell t(\alpha)$. This is essential, since Theorem 1 guarantees the validity of PDIs only when the Farkas certificate is nonnegative.

Corollary 1. *Let $\ell \in \mathcal{K}$, $t \in T$, and $\alpha \in \mathbb{R}^n$. If $\mathcal{Q}^{\ell t}$ is bounded and nonempty, and $\alpha \neq A^{\ell t}_{i\cdot}$ for all $i \in [q + q_t]$, then there exists unique $\bar{x} \in \mathcal{Q}^{\ell t}$ and unique $\bar{v}^t \in \mathcal{D}^{\ell t}(\alpha)$ such that $\alpha^\top \bar{x} = \bar{v}^t b^{kt}$.*

Because the supporting certificate is typically unique for fixed (α, t), solving LP-$\ell t(\alpha)$ is unavoidable; however, storing the basis that originally determined each certificate often accelerates reoptimization after small perturbations. The procedure is agnostic to which condition in Lemma 1 failed—it simply recovers, per term, a certificate that supports the given α. This is precisely the result that, as we show next, is required for Theorem 1 to produce PDIs that support the disjunctive hull.

Algorithm 1 Strong PDI Generator

Require: $k, \ell, \{\mathcal{X}^t\}_{t\in T}, \{v^t\}_{t\in T}, \{\mathcal{A}^t\}_{t\in T}$
Ensure: IP-k and $\{\mathcal{X}^t\}_{t\in T}$ induce $\{v^t\}_{t\in T}$; $\{\mathcal{A}^t\}_{t\in T}$ determines $\{v^t\}_{t\in T}$.
Ensure: There exists $t \in T$ such that $\mathcal{A}^t$ is feasible for $\mathcal{Q}^{\ell t}$ and $v^t \neq 0$.
1: $T_f = \{t \in T : v^t \neq 0, (A^{\ell t}_{\mathcal{A}^t\cdot})^{-1} b^{\ell t}_{\mathcal{A}^t} \in \mathcal{Q}^{\ell t}\}$ ▷ Terms meeting Lemma 1 Condition 1
2: $\{\bar{v}^t\}_{t\in T} \leftarrow \{v^t\}_{t\in T}$
3: $(\alpha, \beta) \leftarrow$ Theorem 1(IP-ℓ, $\{\mathcal{X}^t\}_{t\in T_f}, \{v^t\}_{t\in T_f}$) ▷ Find cut coefficients α
4: **for all** $t \in T^\ell_{\text{feas}}$ such that $A^{kt} \neq A^{\ell t}$ or $t \notin T_f$ **do** ▷ Terms violating Lemma 1
5: $\quad \bar{v}^t \leftarrow \arg\max_{y \in \mathcal{D}^{\ell t}(\alpha)} \{y^\top b^{\ell t}\}$ ▷ Reparameterize
6: **end for**
7: $(\alpha, \bar{\beta}) \leftarrow$ Theorem 1(IP-ℓ, $\{\mathcal{X}^t\}_{t\in T^\ell_{\text{feas}}}, \{\bar{v}^t\}_{t\in T^\ell_{\text{feas}}}$) ▷ Regenerate
8: **return** $(\alpha, \bar{\beta})$

Step 2: Regenerate to Yield Supporting PDIs. We leverage Lemma 2 to construct a Farkas certificate that satisfies the conditions of Lemma 1, ensuring that Theorem 1 produces a cut supporting the disjunctive hull. This process is detailed in Algorithm 1. In Line 3, we obtain an initial valid disjunctive cut. For terms risking loss of support (Figures 1a, 1b, and 5), Line 5 computes supporting certificates via Dual-$\ell t(\alpha)$. Finally Line 7 regenerates a disjunctive-hull-supporting cut per Lemma 1, as shown in Theorem 2.

Theorem 2. *Let $k, \ell \in \mathcal{K}$. Let $\{\mathcal{X}^t\}_{t\in T}$ be a disjunction; $\{v^t\}_{t\in T}$ be Farkas multipliers induced by* IP-k *and $\{\mathcal{X}^t\}_{t\in T}$; $\{\mathcal{A}^t\}_{t\in T}$ be a basis determining $\{v^t\}_{t\in T}$; and $\{(\alpha^k, \beta^k)\}_{k\in\mathcal{K}}$ be the associated family of Farkas PDIs. If there exists $t \in T$ such that $\mathcal{A}^t$ is feasible for $\mathcal{Q}^{\ell t}$ and $v^t > 0$, then $(\alpha^\ell, \bar{\beta}^\ell)$ output from Algorithm 1 supports* $\operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^{\ell t})$.

Thus *any* previously generated valid disjunctive inequality with a Farkas certificate can be tightened to support the disjunctive hull.

4 Computational Experiments

We evaluate implementations of Definition 8 and Algorithm 1 on randomly perturbed MIPLIB 2017 instances, comparing against Balas and Kazachkov [4] and a baseline in which no disjunctive cuts are generated. All runs use standard MILP solvers on a shared server cluster; details below. Bold terms match figure legends and table headers.

Test Set Generation. From **Base** instances $\mathcal{B}$ (presolved MIPLIB-2017; solve time $\geq$ 10s; $\leq$ 5,000 variables and constraints), we build **Test** sets indexed by $(k, u, \theta) \in \mathcal{B} \times \{A, b, c\} \times \{.5, 2\}$, where **Element** $= u$ and **Degree** $= \theta$. Each instance ℓ in (k, u, θ) is produced by Algorithm 3, which rotates u^k by angle θ. For each (k, u, θ) we iterate until we obtain 5 instances, hit 1,000 attempts, or reach 4 hours; hence test-set sizes vary by base instance. We use these test sets to approximate the diversity of possible MILP sequences while avoiding decomposition-specific or real-time–specific structure, enabling a general assessment of our methods.

Methods Compared. Let d denote the number of **Terms** in the disjunction used for cut generation. For each (k, u, θ) and instance ℓ, we augment the root cut pool with:

- **VPC**: $\mathcal{V}$-polyhedral disjunctive cuts [4];
- **PDI**: PDIs via Definition 8;
- **S-PDI**: disjunctive-hull-supporting PDIs via Algorithm 1 (skipped when $u = c$, since Lemma 1 implies Theorem 1 already supports);
- **Default**: default solver (no disjunctive cuts).

For PDI and S-PDI, we reuse the Farkas certificate $\{v^t\}_{t\in T}$ and its determining basis $\{\mathcal{A}^t\}_{t\in T}$ and disjunction $\{\mathcal{X}^t\}_{t\in T}$ obtained when generating disjunctive cuts for IP-k. In what follows, we refer to the disjunction employed by PDI and S-PDI as the PDI disjunction (PD) and to the disjunction employed by VPC as the VPC disjunction (VD).

Software and Hardware Dependencies. All experiments use Gurobi 10 with a root-node callback for adding cuts. We leverage the VPC C++ library [1] as the implementation for Balas and Kazachkov [4, Alg. 1], which uses CBC 2.10 to extract disjunctions from partial branch-and-bound trees. Runs execute on a shared cluster (16 AMD Opteron 6128); each job uses 2 CPUs and up to 15 GB RAM. Time limits are 3,600 s for disjunctive-cut generation and 3,600 s for solving.

4.1 Strength of Root Relaxations

In this section, we show that PDIs effectively tighten root relaxations. We analyze Tables 1 to 3 (see Appendix C) to compare cut strength and root-node efficiency across generators. The "Root Node" columns of Table 4 report the counts of unique base and test instances where root-node cut generation succeeded for all combinations of degree, term, and method.

Root LP with only disjunctive cuts (Table 1). As expected, each relaxation is bounded by the dual bound of its generating disjunction. VPC gives the strongest bounds, followed by S-PDI then PDI—consistent with design: VPC optimizes per instance; S-PDI and PDI reuse bases from prior instances, with S-PDI tighter because it enforces hull support.

Lower perturbation and larger disjunctions yield stronger parametric cuts (bases closer to optimality; tighter relaxations), but overall root-bound gains remain modest. This is expected: many disjunctive cuts are most effective deeper in the tree and are limited by the strength of the disjunctive relaxation. In our set, instances with strong relaxations were often excluded because they were too large, could not be randomly perturbed, or failed to produce non-parametric disjunctive cuts for the base instance, leaving no certificate for our methods to parameterize. We would expect the gap between PDI and S-PDI to widen with greater perturbation and larger disjunctions, but this pattern is not uniform. It

Table 1: Average percent integrality gap closed by disjunctive cuts alone and implied by their generating disjunctions

Degree	Terms	Element	VD	PD	VPC	S-PDI	PDI
0.5	4	matrix	5.96	2.67	3.38	1.90	1.90
		objective	5.37	4.74	3.60	–	3.18
		rhs	8.58	3.69	3.32	2.23	2.23
	64	matrix	12.61	7.88	5.94	3.51	3.31
		objective	13.70	12.09	7.88	–	6.21
		rhs	17.66	9.99	11.16	6.47	5.21
2	4	matrix	8.33	2.53	4.30	1.10	1.10
		objective	6.06	4.35	4.21	–	2.83
		rhs	18.63	3.00	3.14	0.48	0.48
	64	matrix	16.26	5.75	7.43	1.18	0.44
		objective	15.59	10.87	8.52	–	5.22
		rhs	27.38	7.31	9.13	1.16	1.08

does appear, along with larger root-gap gains, when the underlying disjunctive relaxation is strong Table 5 and appendix C, suggesting the irregularities reflect limits of the relaxation rather than of the parametric methods.

Table 2: Average percent integrality gap closed after root node processing

Degree	Terms	Element	VPC	S-PDI	PDI	Default
0.5	4	matrix	73.26	73.25	73.62	73.34
		objective	61.21	–	61.20	60.07
		rhs	51.76	48.47	49.65	50.20
	64	matrix	73.58	73.59	73.40	73.42
		objective	62.05	–	61.69	60.51
		rhs	53.47	53.02	52.69	49.72
2	4	matrix	80.24	80.39	80.41	80.40
		objective	61.84	–	62.34	61.06
		rhs	52.80	52.45	52.89	53.77
	64	matrix	80.61	80.24	79.39	80.12
		objective	62.68	–	62.39	61.14
		rhs	56.35	53.47	53.17	53.91

Root LP after full root processing (Table 2). Adding disjunctive cuts improves the root gap relative to Default, as in Balas and Kazachkov [4]. The strength ordering broadly persists but is less pronounced. Interactions with default root cut generation can blur differences and occasionally make nominally stronger disjunctive cuts yield a similar or weaker overall bound on this test set. As before, larger gains in root relaxation strength appear when the disjunctive relaxations themselves are stronger, as illustrated in Table 6 of Appendix C.

Table 3: Average time (s) to process root node

Degree	Terms	Element	VPC	S-PDI	PDI	Default
0.5	4	matrix	1.394	0.480	0.435	0.314
		objective	1.893	–	0.601	0.572
		rhs	0.690	0.297	0.300	0.275
	64	matrix	11.878	2.135	1.061	0.336
		objective	19.130	–	1.309	0.530
		rhs	6.428	0.744	0.549	0.313
2	4	matrix	2.450	0.377	0.373	0.296
		objective	1.758	–	0.631	0.533
		rhs	0.794	0.248	0.234	0.239
	64	matrix	9.564	1.704	1.211	0.288
		objective	19.378	–	1.408	0.540
		rhs	4.358	0.546	0.515	0.234

Table 4: Unique test instance and total experiment count by setup

Element	Root Node		Overall		Overall > 20	
	Base	Test	Base	Experiments	Base	Experiments
Matrix	89	256	89	460	24	51
Objective	106	578	106	1357	30	126
RHS	59	133	59	250	8	21

Root processing time (Table 3). Runtimes reverse the strength ranking: VPC > S-PDI > PDI > Default (slow to fast). This mirrors computational burden: VPC solves cold-started LPs; S-PDI solves warm-started LPs; PDI uses fixed matrix operations; Default adds no disjunctive cuts. Processing time increases with the number of disjunctive terms, consistent with cut generation complexity scaling in disjunction size.

Takeaway. The methods form a near Pareto frontier trading root-relaxation strength for time: VPC (strongest, slowest), followed by S-PDI and PDI, then finally Default (weakest, fastest). Because overall performance depends on both root strength and generation cost, maintaining multiple generators offers useful flexibility. The next section highlights how combining multiple cut generators with higher-term disjunctions can improve downstream solver performance.

4.2 Overall Performance

In this section, we show that PDIs outperform both VPCs and Gurobi with default settings. We analyze Figures 2 to 4 to compare overall branch-and-cut solve time of Gurobi configured with VPC, S-PDI, PDI, and Default. In each figure, the left panel is a performance profile of S-PDI and PDI relative to VPC over all test instances, where a value of -0.5 denotes a 50% speedup and +0.5

a 50% slowdown on instances solved by all methods. The right panel compares VPC, S-PDI, and PDI against Default, restricted to instances where Default required at least 20 minutes. "Best" denotes the virtual best over all configurations. The "Overall" and "Overall > 20" columns of Table 4 report, respectively, the counts of unique base instances and total experiments that reached optimality, and the subset for which Default took > 20 minutes. These need not match the count of test instances in "Root Node" because (a) each test instance is rerun for every degree, (b) some runs that finish root-node processing still time out before optimality, and (c) some base instances fail to yield nonparametric disjunctive cuts for certain (degree, term) pairs—so the parametric methods cannot start—whereas many runs that do receive certificates subsequently reach optimality.

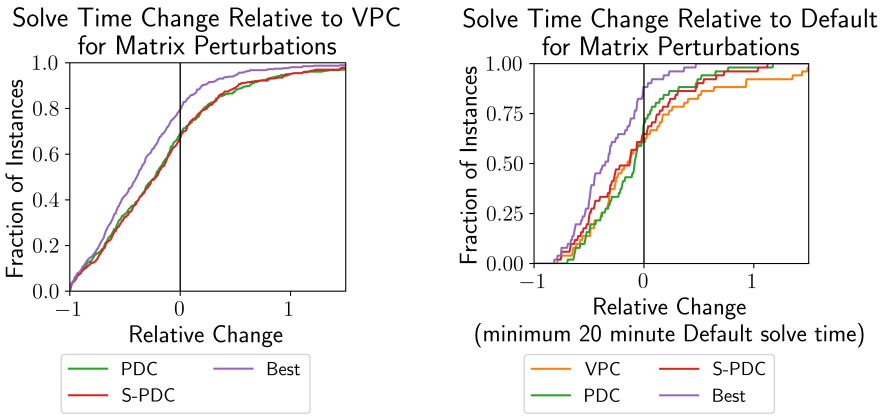


Fig. 2: For matrix perturbations, PDIs often outperform VPCs (left), and the best disjunctive cut generator almost always beats default Gurobi (right)

Matrix perturbations: relative to VPC. In Figure 2 (left), both S-PDI and PDI beat VPC on roughly 65% of instances. The virtual best improves on VPC in about 80% of cases, indicating complementary strengths between S-PDI and PDI. This aligns with Tables 1 to 3: S-PDI and PDI trade a small amount of cut strength for markedly lower generation cost, yielding better end-to-end times.

Matrix perturbations: relative to Default. In Figure 2 (right), VPC, S-PDI, and PDI each outperform Default on approximately 60–65% of instances with solve times above 20 minutes. The virtual best wins on nearly 90% of these cases, with about half solving at least twice as fast. This points to a usual branch-and-bound tradeoff: stronger root cuts increase root processing time, but they tighten the relaxation globally. On harder, long-running instances, implicit pruning from the tighter relaxation can accumulate and the upfront cost is amortized, creating opportunity to lower total solve time.

Objective and right-hand side perturbations. The same patterns hold for objective-coefficient and right-hand-side perturbations: S-PDI and PDI compare favorably

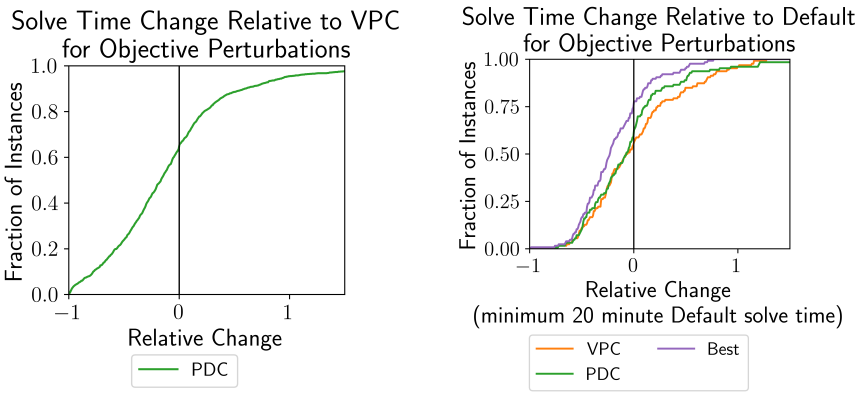


Fig. 3: PDIs often outperform VPCs (left) and default Gurobi (right) for objective perturbations

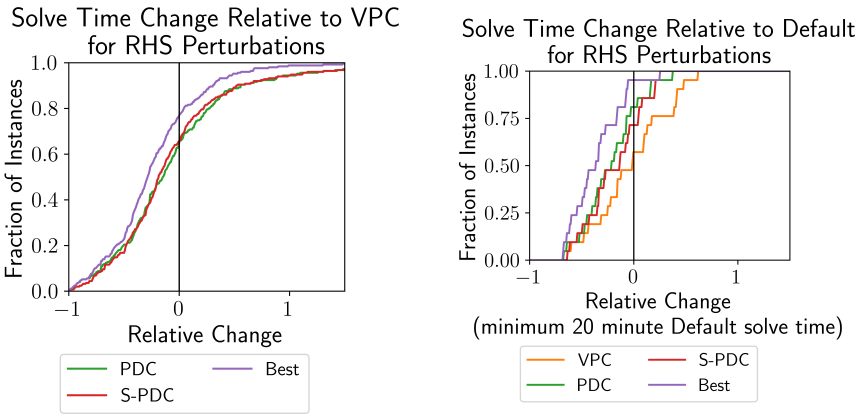


Fig. 4: PDIs often outperform VPCs and default Gurobi for right-hand side perturbations

to VPC on the left panels, and all three disjunctive generators improve on Default for long-running instances on the right panels (Figures 3 and 4, Appendix C). These results reinforce that the observed trends are robust across perturbation types and not specific to the constraint matrix.

5 Conclusion

This paper provides theory on how to generate PDIs that remain valid (and supporting) across related MILPs without solving a cut-generating LP; dual solutions of the tightening LPs certify support for individual terms and, when fed into Theorem 1, yield cuts that provably support the disjunctive hull. In our experiments, strengthened and unstrengthened PDIs tighten the default root relaxation while retaining much of the speed advantage over Balas and Kazachkov

[4]. Within branch and cut, our best parametric generator consistently outperforms Balas and Kazachkov [4] in total solve time, and the strongest disjunctive generator beats default Gurobi on most instances exceeding 20 minutes.

Open questions remain, notably how to choose the most effective inequality per instance; under matrix perturbations, deriving the supporting certificate from the original (nonparametric) cut may further help.

Extensions include learning-guided selection of generators, comparing our disjunction-based warm start with an explicit MILP warm start using the same disjunction, and applying our strengthening routine to other costly families (e.g., bilevel interdiction cuts) via Algorithm 1.

Methodologically, we provide a pathway to parameterize disjunctive inequalities and recover hull support as needed. Practically, we offer guidance for warm-starting the dual side—especially the cut pool—offering reduced solve times in sequential and decomposed settings and highlighting gains from exploiting structure across sequences of related problems.

Bibliography

[1] V-polyhedral Disjunctive Cuts. https://github.com/spkelle2/vpc (2023)

[2] COIN-OR Branch and Cut. https://github.com/coin-or/Cbc (2024)

[3] Balas, E.: Disjunctive programming. Ann. Discrete Math. **5**, 3–51 (1979), URL https://www.sciencedirect.com/science/article/pii/S016750600870342X

[4] Balas, E., Kazachkov, A.M.: $\mathcal{V}$-polyhedral disjunctive cuts (2025), URL https://doi.org/10.1007/s10107-025-02227-y

[5] Barnhart, C., Johnson, E.L., Nemhauser, G.L., Savelsbergh, M.W.P., Vance, P.H.: Branch-and-price: column generation for solving huge integer programs. Oper. Res. **46**(3), 316–329 (1998), ISSN 0030-364X,1526-5463, URL https://doi.org/10.1287/opre.46.3.316

[6] Beasley, J.E.: Lagrangian relaxation, p. 243–303. John Wiley & Sons, Inc., USA (1993), ISBN 0470220791

[7] Becu, B., Dey, S.S., Qiu, F., Xavier, A.S.: Approximating the Gomory mixed-integer cut closure using historical data (2024), URL https://arxiv.org/abs/2411.15090

[8] Bixby, R., Rothberg, E.: Progress in computational mixed integer programming—a look back from the other side of the tipping point. Ann. Oper. Res. **149**, 37–41 (2007), URL https://doi.org/10.1007/s10479-006-0091-y, history of integer programming: distinguished personal notes and reminiscences

[9] Chen, B., Küçükyavuz, S., Sen, S.: Finite disjunctive programming characterizations for general mixed-integer linear programs. Oper. Res. **59**(1), 202–210 (2011), URL https://doi.org/10.1287/opre.1100.0882

[10] Chételat, D., Lodi, A.: Continuous cutting plane algorithms in integer programming. Oper. Res. Lett. **51**(4), 439–445 (2023), ISSN 0167-6377, URL https://www.sciencedirect.com/science/article/pii/S0167637723001074

[11] Deza, A., Khalil, E.B.: Machine learning for cutting planes in integer programming: A survey. In: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, p. 6592–6600, IJCAI-2023, International Joint Conferences on Artificial Intelligence Organization (Aug 2023), URL http://dx.doi.org/10.24963/IJCAI.2023/739

[12] Dragotto, G., Clarke, S., Fisac, J.F., Stellato, B.: Differentiable cutting-plane layers for mixed-integer linear optimization (2023), URL https://arxiv.org/abs/2311.03350

[13] Duran, M.A., Grossmann, I.E.: An outer-approximation algorithm for a class of mixed-integer nonlinear programs. Math. Program. **36**(3), 307–339 (1986), ISSN 0025-5610,1436-4646, URL https://doi.org/10.1007/BF02592064

[14] Fallah, S., Ralphs, T.K., Boland, N.L.: On the relationship between the value function and the efficient frontier of a mixed integer linear optimization problem (2024), URL https://arxiv.org/abs/2303.00785

[15] Farkas, J.: Theorie der einfachen Ungleichungen. J. Reine Angew. Math. **124**, 1–27 (1902), URL https://doi.org/10.1515/crll.1902.124.1

[16] Gasse, M., Chételat, D., Ferroni, N., Charlin, L., Lodi, A.: Exact combinatorial optimization with graph convolutional neural networks. In: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada, pp. 15554–15566 (2019), URL https://proceedings.neurips.cc/paper/2019/hash/d14c2267d848abeb81fd590f371d39bd-Abstract.html

[17] Güzelsoy, M.: Dual Methods in Mixed Integer Linear Programming. PhD, Lehigh University (2009), URL http://coral.ie.lehigh.edu/~ted/files/papers/MenalGuzelsoyDissertation09.pdf

[18] Hassanzadeh, A., Ralphs, T.: A Generalization of Benders' Algorithm for Two-Stage Stochastic Optimization Problems with Mixed Integer Recourse. Tech. Rep. 14T-005, COR@L Laboratory, Lehigh University (2014), URL http://coral.ie.lehigh.edu/~ted/files/papers/SMILPGenBenders14.pdf

[19] Kazachkov, A.M., Balas, E.: Monoidal strengthening of simple $\mathcal{V}$-polyhedral disjunctive cuts (3 2025), URL https://doi.org/10.1007/s10107-024-02185-x

[20] Muñoz, G., Paat, J., Xavier, A.S.: Compressing branch-and-bound trees. In: Integer programming and combinatorial optimization, Lecture Notes in Comput. Sci., vol. 13904, pp. 348–362, Springer, Cham (2023), ISBN 978-3-031-32725-4; 978-3-031-32726-1, URL https://doi.org/10.1007/978-3-031-32726-1_25

[21] Nair, V., Bartunov, S., Gimeno, F., von Glehn, I., Lichocki, P., Lobov, I., O'Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., Addanki, R., Hapuarachchi, T., Keck, T., Keeling, J., Kohli, P., Ktena, I., Li, Y., Vinyals, O., Zwols, Y.: Solving mixed integer programs using neural networks (2021), URL https://arxiv.org/abs/2012.13349

[22] Nazari, M., Oroojlooy, A., Snyder, L., Takac, M.: Reinforcement learning for solving the vehicle routing problem. In: Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (eds.) Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc. (2018), URL https://proceedings.neurips.cc/paper_files/paper/2018/file/9fb4651c05b2ed70fba5afe0b039a550-Paper.pdf

[23] Perregaard, M., Balas, E.: Generating cuts from multiple-term disjunctions. In: Integer programming and combinatorial optimization (Utrecht, 2001), Lecture Notes in Comput. Sci., vol. 2081, pp. 348–360, Springer, Berlin (2001), ISBN 3-540-42225-0, URL https://doi.org/10.1007/3-540-45535-3_27

[24] Tahernejad, S., Ralphs, T.K., DeNegre, S.T.: A branch-and-cut algorithm for mixed integer bilevel linear optimization problems and its im-

plementation. Math. Program. Comput. **12**(4), 529–568 (2020), ISSN 1867-2949,1867-2957, URL https://doi.org/10.1007/s12532-020-00183-6

[25] Xavier, A.S., Qiu, F., Ahmed, S.: Learning to solve large-scale security-constrained unit commitment problems. INFORMS J. Comput. **33**(2), 739–756 (2021), ISSN 1091-9856,1526-5528, URL https://doi.org/10.1287/ijoc.2020.0976

A Proofs

Proof of Theorem 1

Proof. Let $\gamma^t := v^t A^{kt}$ and $\gamma_0^t := v^t b^{kt}$ for all $t \in T$. We will show that $\alpha^\top x \geq \beta$ is valid for $\mathcal{Q}^{kt}$ for a fixed (arbitrary) index $t \in T$. We have that $\gamma^t x \geq \gamma_0^t$ is valid for $\mathcal{Q}^{kt}$. Then, if $x \in \mathcal{Q}^{kt}$, since we have nonnegativity on the variables ($\mathcal{Q}^{kt} \subseteq \mathbb{R}^n_{\geq 0}$), it follows that,

$$\alpha^\top x = \sum_{j\in[n]} \max_{t'\in T}\{\gamma_j^{t'}\}x_j \geq \sum_{j\in[n]} \gamma_j^t x_j \geq \gamma_0^t \geq \min_{t'\in T}\{\gamma_0^{t'}\} = \beta.$$

Hence, $\alpha^\top x \geq \beta$ is valid for $\bigcup_{t\in T} \mathcal{Q}^{kt} \supseteq \mathcal{S}^k \cap \mathcal{P}_D^k$.

Proof of Lemma 1

Proof. Let $t' = \arg\min_{t\in T}\{v^t b^{\ell t}\}$. Our hypothesis provides $v^t A^{\ell t} = v^t A^{kt} = \alpha$ for all $t \in T$, which implies that $\max_{t\in T}\{v^t A^{\ell t}\} = v^{t'} A^{\ell t'}$. Applying Theorem 1 yields $(\alpha, \beta) = (\max_{t\in T}\{v^t A^{\ell t}\}, \min_{t\in T}\{v^t b^{\ell t}\}) = (v^{t'} A^{\ell t'}, v^{t'} b^{\ell t'})$, a valid inequality for $\operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^{\ell t})$. Assumptions on $\mathcal{A}^t$ imply that $\alpha = v^{t'}_{\mathcal{A}^t} A^{\ell t'}_{\mathcal{A}^t \cdot}$, $v^{t'}_{\mathcal{A}^t} = \alpha(A^{\ell t'}_{\mathcal{A}^t \cdot})^{-1}$, $v^{t'}_{\mathcal{A}^t} b^{\ell t'}_{\mathcal{A}^t} = \alpha(A^{\ell t'}_{\mathcal{A}^t \cdot})^{-1} b^{\ell t'}_{\mathcal{A}^t}$, and $(A^{\ell t'}_{\mathcal{A}^t \cdot})^{-1} b^{\ell t'}_{\mathcal{A}^t} \in \mathcal{Q}^{\ell t}$. Since $\beta = v^{t'}_{\mathcal{A}^t} b^{\ell t'}_{\mathcal{A}^t}$, (α, β) supports $\mathcal{Q}^{\ell t'}$. Given $\mathcal{P}_D^\ell = \operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^{\ell t})$ and is convex, (α, β) supports it, too.

Proof of Lemma 2

Proof. Let $\bar{x} \in \arg\min_{x\in\mathcal{Q}^{\ell t}}\{\alpha^\top x\}$ and $\bar{v}^t \in \arg\max_{v^t\in\mathcal{D}^{\ell t}(\alpha)}\{v^t b^{\ell t}\}$. Since $\bar{x}$ optimal for LP-$\ell t(\alpha)$, we have that $\alpha^\top x \geq \alpha^\top \bar{x}$ for all $x \in \mathcal{Q}^{\ell t}$, which means that $\alpha, \alpha^\top \bar{x}$ supports $\mathcal{Q}^{kt}$. Since $\bar{v}^t$ is feasible for Dual-$\ell t(\alpha)$, we have that $\bar{v}^t A^{\ell t} = \alpha$. By stong duality, $\bar{v}^t b^{\ell t} = \alpha^\top \bar{x}$, which means that $\bar{v}^t$ is a certificate for the cut $(\alpha, \alpha^\top \bar{x})$ and is thus supporting.

Proof of Corollary 1

Proof. Lemma 2 provides existence of $\bar{x}$ and $\bar{v}^t$. Since there does not exist $i \in [q + q_t]$ such that $\alpha = A^{kt}_{i\cdot}$, pivoting from $\bar{x}$ to another $x \in \mathcal{Q}^{kt}$ would violate dual feasibility. Therefore, $\bar{x} \in \mathcal{Q}^{kt}$ and $\bar{y} \in \mathcal{D}^{\ell t}(\alpha)$ such that $\alpha^\top \bar{x} = \bar{y}^\top b^{kt}$ are unique.

Proof of Theorem 2

Proof. Define T_f as in Line 1, which is nonempty due to the existence of $t \in T$ such that $\mathcal{A}^t$ is feasible for $\mathcal{Q}^{\ell t}$ and $v^t > 0$. As prescribed in Line 2, let $\{\bar{v}^t\}_{t\in T} = \{v^t\}_{t\in T}$, and, additionally, let $\{\bar{\mathcal{A}}^t\}_{t\in T} = \{\mathcal{A}^t\}_{t\in T}$. By Lemma 1,

Line 3 provides $\alpha = \bar{v}^t A^{\ell t}$ for all $t \in T_c$. Consider $t \in T^\ell_{\text{feas}} \setminus T_c$. Update $\bar{\mathcal{A}}^t$ such that $A^{\ell t}_{\bar{\mathcal{A}}^t \cdot} \bar{x}^t = b^{\ell t}_{\bar{\mathcal{A}}^t}$ for $\bar{x}^t \in \arg\max_{x \in \mathcal{Q}^{\ell t}} \{\alpha^\top x\}$. Lemma 2 proves in Line 5 that $\bar{v}^t \in \mathbb{R}^{1 \times n}_{\geq 0}$ and $\alpha = \bar{v}^t A^{\ell t}$. Since $\bar{x}^t$ and $\bar{v}^t$ are dual to each other, we have that $\{i \in [q + q_t] : \bar{v}^t_i > 0\} \subseteq \bar{\mathcal{A}}^t$. Therefore, we have that IP-ℓ and $\{\mathcal{X}^t\}_{t \in T^\ell_{\text{feas}}}$ induce $\{\bar{v}^t\}_{t \in T^\ell_{\text{feas}}}$, $\{\bar{\mathcal{A}}^t\}_{t \in T}$ determines $\{\bar{v}^t\}_{t \in T^\ell_{\text{feas}}}$ and, for all $t \in T^\ell_{\text{feas}}$, $\bar{\mathcal{A}}^t$ is feasible for $\mathcal{Q}^{\ell t}$. thus, Lemma 1 provides $(\alpha, \bar{\beta})$ in Line 7 supports $\operatorname{cl}\operatorname{conv}(\cup_{t \in T^\ell_{\text{feas}}} \mathcal{Q}^{\ell t}) = \operatorname{cl}\operatorname{conv}(\cup_{t \in T} \mathcal{Q}^{\ell t})$.

B Algorithms

Algorithm 2 Find Degree

Require: u^k, u_{new}

1: angle_difference $\leftarrow \cos^{-1}\left(\frac{u^k \cdot u_{\text{new}}}{\|u^k\| \|u_{\text{new}}\|}\right)$ ▷ Angle between vectors
2: norm_difference $\leftarrow \left|\frac{\|u^k\| - \|u_{\text{new}}\|}{\|u^k\|}\right|$ ▷ Relative change in vector norms
3: **return** max{angle_difference, norm_difference}

Algorithm 3 Find Perturbation

Require: u^k, θ ▷ starting vector, desired perturbation
1: $u^k \leftarrow \texttt{toVector}(u^k)$ if $u^k \in \mathbb{R}^{q \times n}$ else u^k ▷ Flatten to vector if needed
2: $u_{\text{final}}, \epsilon \leftarrow$ NULL, 1
3: **while** $\epsilon \geq 10^{-6}$ and $u_{\text{final}} ==$ NULL **do** ▷ until tolerance met or perturbed vector found
4: $u_{\text{new}}, u_{\text{prev}} \leftarrow u^k$, NULL
5: **while** Find Degree$(u^k, u_{\text{new}}) < \theta$ **do** ▷ Until desired perturbation reached
6: $u_{\text{prev}} \leftarrow u_{\text{new}}$
7: $i \leftarrow \texttt{random}([|u|])$ ▷ Select random element
8: $u_{\text{new}}[i] \leftarrow u_{\text{new}}[i] + \texttt{random}([-\epsilon, \epsilon])$ ▷ Randomly perturb by $\pm\epsilon$
9: **end while**
10: **if** $u_{\text{prev}} \neq u^k$ **then** ▷ If an iteration produced a vector within the desired perturbation
11: $u_{\text{final}} \leftarrow u_{\text{prev}}$
12: **end if**
13: $\epsilon \leftarrow \epsilon/2$
14: **end while**
15: **return** $\texttt{toMatrix}(u_{\text{final}})$ if $u_{\text{final}} \in \mathbb{R}^{qn}$ else u_{final} ▷ Return as matrix if needed

C Additional Tables and Figures

Figure 5 illustrates the second failure mode of Lemma 1 Condition 1: the PDI may not support the disjunctive hull when perturbation induces feasibility for a disjunctive term. As mentioned in Section 2, $v^t = 0$ for initially infeasible disjunctive terms, necessitating calculation of a supporting certificate.

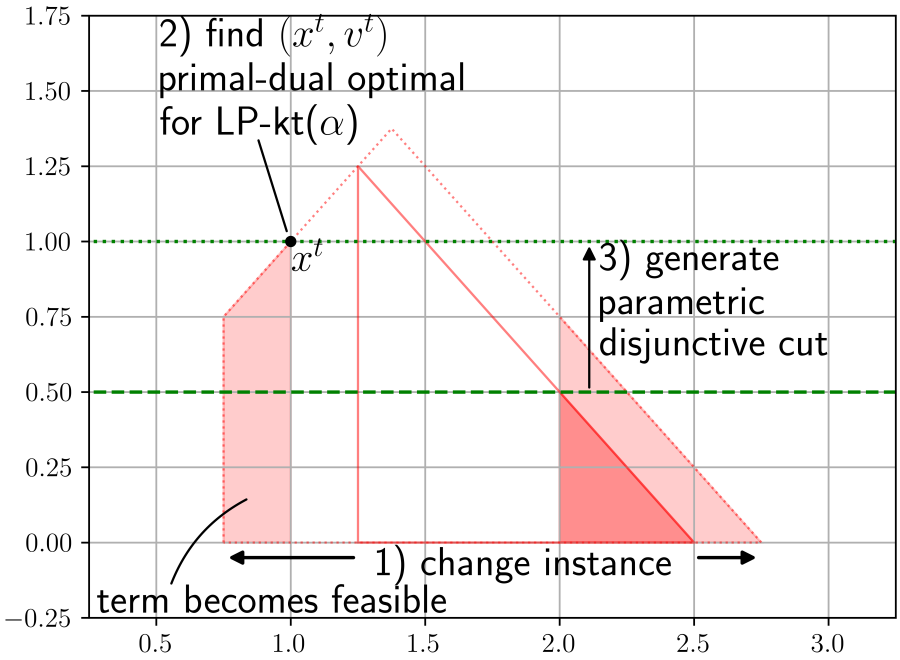


Fig. 5: For perturbation-induced feasible terms

Tables 5 to 7 highlight the strength differences of our disjunctive cut generators compared to their generating disjunctions and solver defaults, which are made possible by the presence of a strong disjunctive relaxation.

Table 5 mirrors Table 1, reporting the percentage of integrality gap closed over 20 random perturbations of `bm23.mps`. In this case the separation of S-PDI from PDI is clearer for larger disjunctions and higher perturbation levels—precisely where parameterization has more scope to weaken. We also see the expected monotone effects: more terms and smaller perturbations yield stronger parametric cuts, and disjunctive cuts are strong overall. Apparent exceptions arise when the underlying disjunctive relaxation is stronger for the PDI disjunction than for the VPC disjunction (e.g., PD > VD for most right-hand-side perturbations), in which case S-PDI and PDI can surpass VPC.

Table 6 parallels Table 2, reporting the percentage of integrality gap closed by the LP relaxation after root processing for strong disjunctive relaxations generated from random perturbations of `bm23.mps`. Given the strength of the relaxation and the clear separation among generators in Table 5, the differences between VPC, S-PDI, PDI, and Default are more pronounced here—offering a cleaner extension of the patterns seen at the "cuts only" level than in the aggregate tables. Deviations from monotonicity arise when the underlying disjunctive relaxation favors the PDI disjunction over the VPC disjunction (e.g., PD > VD) and for 4-term disjunctions where disjunctive cuts fail to tighten the relaxation near the optimal region under the default cuts. Overall, the expected trends

Table 5: Average percent integrality gap closed by disjunctive cuts alone and implied by their generating disjunctions, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from `bm23.mps`

Degree	Terms	Element	VD	PD	VPC	S-PDI	PDI
0.5	4	matrix	14.78	14.78	14.78	14.63	14.62
		objective	14.63	14.63	14.63	–	14.63
		rhs	13.85	14.60	13.85	14.22	14.22
	64	matrix	71.00	71.35	69.05	67.23	66.46
		objective	68.83	71.48	66.74	–	69.37
		rhs	69.03	70.93	66.91	68.17	64.77
2	4	matrix	13.38	14.33	13.22	13.00	12.90
		objective	14.79	14.79	14.74	–	14.76
		rhs	11.77	13.27	11.61	12.16	12.16
	64	matrix	70.44	71.38	67.68	49.42	28.17
		objective	66.90	70.41	64.82	–	67.60
		rhs	69.49	63.82	66.88	60.35	44.03

persist: more terms and smaller perturbations yield larger root-gap reductions, and stronger generators deliver greater improvements.

Table 7 parallels Table 3, reporting root processing times under strong disjunctive relaxations from random perturbations of `bm23.mps`. Since cut-generation complexity does not depend on the strength of the disjunctive relaxation, the ordering of runtimes is essentially unchanged: VPC is slowest, followed by S-PDI, then PDI, with Default fastest. These results reinforce the expected trade-off between generator strength and processing time, with a clear negative association between strength and speed.

Table 6: Average percent integrality gap closed after root node processing, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from `bm23.mps`

Degree	Terms	Element	VPC	S-PDI	PDI	Default
0.5	4	matrix	42.63	40.62	40.48	42.16
		objective	39.18	–	40.89	39.23
		rhs	43.19	41.49	42.00	43.87
	64	matrix	70.30	69.06	68.57	42.13
		objective	68.40	–	70.52	39.38
		rhs	68.69	69.44	68.08	42.63
2	4	matrix	39.92	40.10	41.11	38.71
		objective	40.16	–	39.54	40.02
		rhs	42.92	42.49	41.60	41.16
	64	matrix	69.56	56.42	48.44	38.82
		objective	67.09	–	69.28	40.01
		rhs	69.34	62.25	56.53	41.16

Table 7: Time to process root node, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from `bm23.mps`

Degree	Terms	Element	VPC	S-PDI	PDI	Default
0.5	4	matrix	0.227	0.103	0.113	0.134
		objective	0.200	–	0.122	0.105
		rhs	0.213	0.122	0.124	0.162
	64	matrix	1.629	0.405	0.176	0.131
		objective	1.661	–	0.173	0.103
		rhs	1.629	0.214	0.174	0.127
2	4	matrix	0.219	0.143	0.126	0.102
		objective	0.209	–	0.119	0.124
		rhs	0.227	0.135	0.118	0.105
	64	matrix	1.702	0.548	0.214	0.104
		objective	1.633	–	0.176	0.118
		rhs	1.589	0.364	0.198	0.106